

§ 4. 函数

1. 在VS下建立多源程序文件项目

★ 含头文件

★ 不含头文件





§ 4. 函数

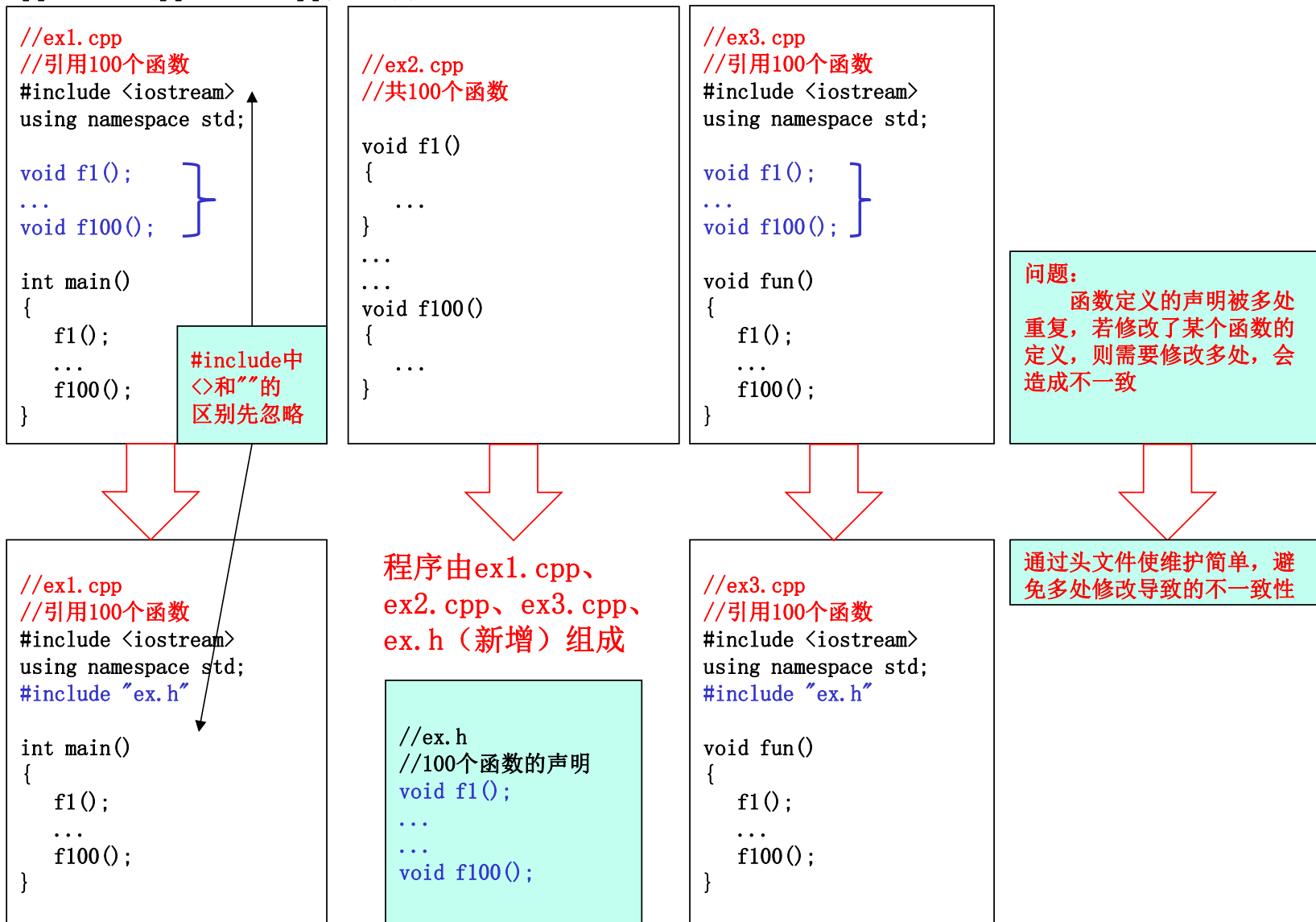
1. 在VS下建立多源程序文件项目
2. 头文件的内容及作用

头文件的内容:

- ★ 结构体类型(struct-后续模块)及类(class-后续模块)的声明
- ★ 函数的声明
- ★ inline函数的定义与实现
- ★ 符号常量的定义及常变量的定义
- ★ 全局变量的extern声明
- ★ 其它需要的头文件



例：程序由ex1.cpp、ex2.cpp、ex3.cpp共同构成

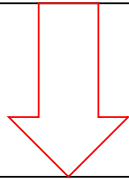




例：程序由ex1.cpp、ex2.cpp共同构成

```
//ex1.cpp
#include <iostream>
using namespace std;
inline void f1()
{
    ...
}

int main()
{
    f1();
    ...
    f1();
}
```



```
//ex1.cpp
#include <iostream>
using namespace std;

#include "ex.h"

int main()
{
    f1();
    ...
    f1();
}
```

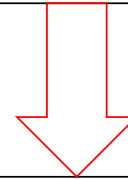
程序由ex1.cpp、
ex2.cpp、ex.h
(新增) 组成

```
//ex.h

inline void f1()
{
    ...
}
```

```
//ex2.cpp
#include <iostream>
using namespace std;
inline void f1()
{
    ...
}

void fun()
{
    f1();
    ...
    f1();
}
```

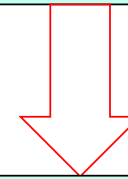


```
//ex2.cpp
#include <iostream>
using namespace std;

#include "ex.h"

void fun()
{
    f1();
    ...
    f1();
}
```

问题：
因为inline函数
必须和调用函数处在
同一个源文件中，
导致多处重复



通过头文件使维护简单，避
免多处修改导致的不一致性

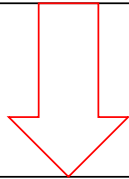


例：程序由ex1.cpp、ex2.cpp共同构成

```
//ex1.cpp
#include <iostream>
using namespace std;

#define pi 3.14159
const int x=10;

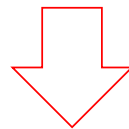
int main()
{
    ...pi...
    ...
    ...x...
}
```



```
//ex1.cpp
#include <iostream>
using namespace std;

#include "ex.h"

int main()
{
    ...pi...
    ...
    ...x...
}
```



程序由ex1.cpp、
ex2.cpp、ex.h
(新增) 组成

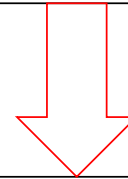
```
//ex.h

#define pi 3.14159
const int x=10;
```

```
//ex2.cpp
#include <iostream>
using namespace std;

#define pi 3.14159
const int x=10;

void fun()
{
    ...pi...
    ...
    ...x...
}
```

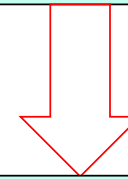


```
//ex2.cpp
#include <iostream>
using namespace std;

#include "ex.h"

void fun()
{
    ...pi...
    ...
    ...x...
}
```

问题：
符号常量及常变量在多处定义，导致重复定义以及维护困难



通过头文件使维护简单，避免多处修改导致的不一致性



例：程序由ex1.cpp、ex2.cpp、ex3.cpp共同构成

```
//ex1.cpp  
//定义全局变量
```

```
#include <iostream>  
using namespace std;
```

```
int x=10;
```

```
int main()  
{  
    ...x...  
}
```

```
//ex2.cpp  
//引用全局变量
```

```
extern int x;  
void f1()  
{  
    ...x...  
}
```

```
void f2()  
{  
    ...x...  
}
```

```
//ex3.cpp  
//引用全局变量
```

```
extern int x;  
void fun()  
{  
    ...x...  
}
```

```
//ex1.cpp  
//定义全局变量
```

```
#include <iostream>  
using namespace std;
```

```
int x=10;
```

```
int main()  
{  
    ...x...  
}
```

```
//ex2.cpp  
//引用全局变量
```

```
#include "ex.h"  
void f1()  
{  
    ...x...  
}
```

```
void f2()  
{  
    ...x...  
}
```

```
//ex3.cpp  
//引用全局变量
```

```
#include "ex.h"  
void fun()  
{  
    ...x...  
}
```

程序由ex1.cpp、
ex2.cpp、ex3.cpp、
ex.h（新增）组成

```
//ex.h  
//全局变量声明
```

```
extern int x;
```



例：程序由ex1.cpp、ex2.cpp、ex3.cpp共同构成

```
//ex1.cpp  
//定义全局变量
```

```
#include <iostream>  
using namespace std;
```

```
int x=10;
```

```
int main()  
{  
    ...x...  
}
```



```
//ex1.cpp  
//定义全局变量
```

```
#include <iostream>  
using namespace std;
```

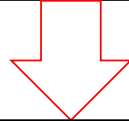
```
#include "ex.h"
```

```
int main()  
{  
    ...x...  
}
```

```
//ex2.cpp  
//引用全局变量
```

```
extern int x;  
void f1()  
{  
    ...x...  
}
```

```
void f2()  
{  
    ...x...  
}
```



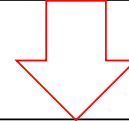
```
//ex2.cpp  
//引用全局变量
```

```
#include "ex.h"  
void f1()  
{  
    ...x...  
}
```

```
void f2()  
{  
    ...x...  
}
```

```
//ex3.cpp  
//引用全局变量
```

```
extern int x;  
void fun()  
{  
    ...x...  
}
```



```
//ex3.cpp  
//引用全局变量
```

```
#include "ex.h"  
void fun()  
{  
    ...x...  
}
```



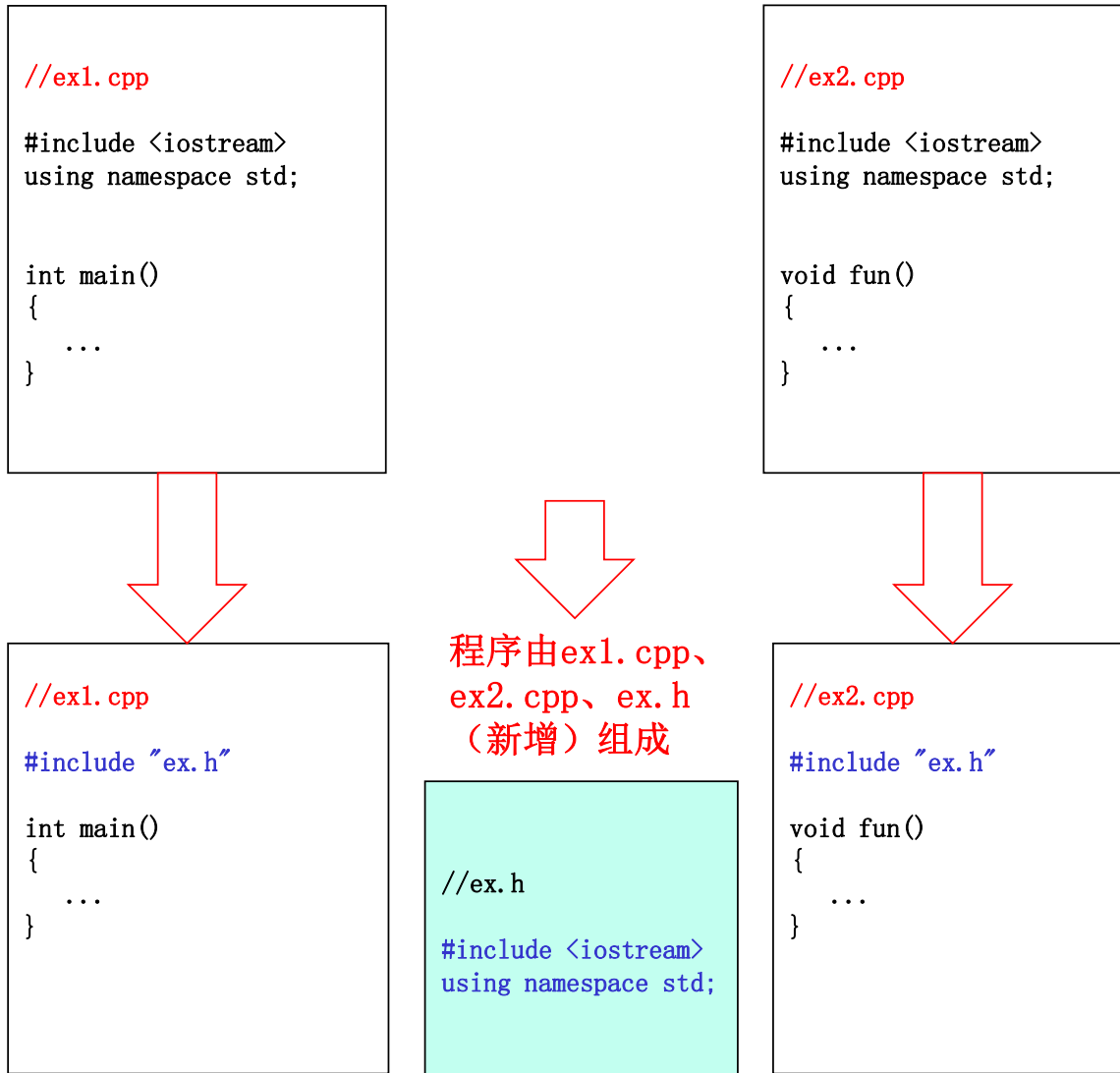
程序由ex1.cpp、
ex2.cpp、ex3.cpp、
ex.h（新增）组成

```
//ex.h  
//全局变量定义  
int x; //错误
```

注:1. 若头文件中包含全局变量
定义, 则被多个文件包含
会导致重复定义
2. 头文件中可包含静态全局/
只读变量, 但不建议
const int x = 10;
static int x = 15;



例：程序由ex1.cpp、ex2.cpp共同构成





§ 4. 函数

1. 在VS下建立多源程序文件项目

2. 头文件的内容及作用

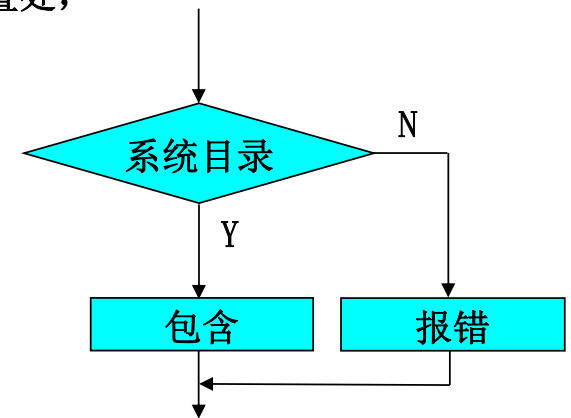
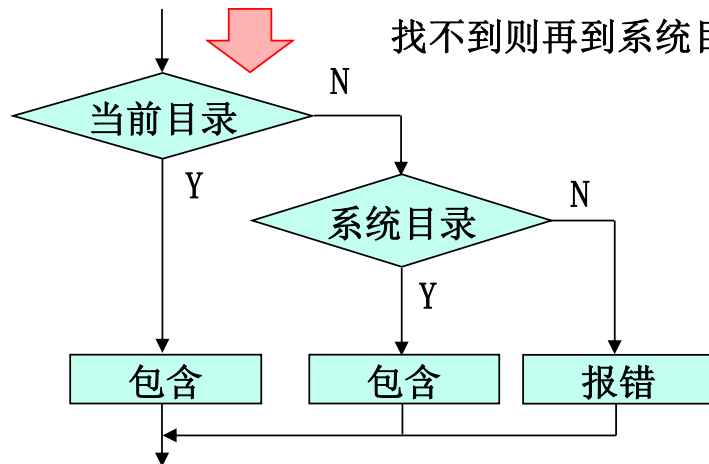
头文件的作用：

- ★ 将编程者需要的在不同源程序文件传递的各种信息归集在一起，方便多次调用以及集中修改
- ★ 在一个源程序文件中包含头文件时，头文件的所有内容会被理解为包含到 `#include` 位置处，编译时（变量的定义及函数作用域等）均当作一个文件进行处理

头文件的包含方式：

`#include <文件名>`：直接到系统目录中寻找，找到则包含进来，找不到则报错

`#include "文件名"`：先在当前目录中寻找，找到则包含进来，找不到则再到系统目录中寻找，找到则包含进来，找不到则报错



VS2026如果缺省安装，则头文件的目录为
64位Windows操作系统：

C:\Program Files\Microsoft Visual Studio\18\Community\VC\Tools\MSVC\xx.xx.xxxxx\include

具体版本号



§ 4. 函数

例:理解<>和""的差别

```
//demo.cpp  
#include <iostream>  
using namespace std;
```

```
int main()  
{  
    cout << "Hello" << endl;  
    return 0;  
}
```

编译正确

```
//demo.cpp  
#include "iostream"  
using namespace std;
```

```
int main()  
{  
    cout << "Hello" << endl;  
    return 0;  
}
```

编译正确

```
//demo.cpp  
#include "iostream"  
using namespace std;
```

```
int main()  
{  
    cout << "Hello" << endl;  
    return 0;  
}
```

编译错误

例: 在当前目录下建立
一个iostream
内容为空

```
//demo.cpp  
#include "iostream"  
using namespace std;
```

```
int main()  
{  
    cout << "Hello" << endl;  
    return 0;  
}
```

编译正确

例: 在当前目录下建立
一个iostream
内容:
#include <iostream>



§ 4. 函数

1. 在VS下建立多源程序文件项目
2. 头文件的内容及作用
3. C++的标准库及头文件

C++包含系统头文件的两种形式:

`#include <math.h>` : C形式 (适用于.c和.cpp两种形式的源程序文件)

`#include <cmath>` : C++形式(仅适用于.cpp形式的源程序文件)

两种方式都是指编译系统的include目录的math.h

VS2026如果缺省安装, 则头文件的目录为

64位Windows操作系统:

C:\Program Files\Microsoft Visual Studio\18\Community\VC\Tools\MSVC\xx.xx.xxxxx\include

具体版本号